Amanu: Conceptualizing a Simple Kapampangan Generative Artificial Intelligence Language Model

Caranto, Crisiane Josef A.

Discrete Mathematics and Structures

Case Study

I. Conceptualizing the Model

In the current iteration of the artificial intelligence (AI) innovations, large language models (LLMs) have become integrated into the digital landscape of various industries. Models such as OpenAI's ChatGPT, Google's Gemini, and Anthropic's Claude are trained on vast datasets, trained to process and understand human natural language and programming code.

From this, mathematical models are used by these LLMs to understand language. As words from a language are distinct from one another, they are often described as *discrete* structures. In relation to this, discrete structures can also be used within generative AI models such as LLMs.

These are a type of AI models that generate an output from probability distributions learned from the dataset from which they are trained. Thus, a language input such as "hello" may be mapped out to the word "world" to produce the output "hello world" if the probability of the word "world" is higher than any candidate word.

In the context of discrete mathematics and structures, the notions on AI language models can be represented through logic, sets and set operations, functions, relations, and matrices.

The case study then proposes a simple language model, named Amanu, that will predict a word from a set of Kapampangan words based on an input syllable by the user. This prediction will be based on matrix vectors that are calculated to obtain various probability values for each word where the highest is outputted by Amanu.

Through this case study, not only will fundamental discrete mathematics concepts be utilized in a practical manner, but basic understanding of AI knowledge and promotion of Kapampangan heritage will also be highlighted.

II. Defining the Sets

In the Kapampangan language, focusing on the alphasyllabary writing system of Kulitan, it has 11 base alphasyllabic consonant sounds and five vowel sounds. Defining them in a set, set C will be the consonant sounds and set V will be the vowel sounds.

$$C = \{ba, ma, pa, sa, la, na, ga, ka, nga, ta, da\}$$

$$V = \{a, i, u, e, o\}$$

$$F = C \cup V$$

From this, each syllable will each have three Kapampangan words starting with the syllable as a base training for this simple generative language model. These will be set Y containing the dictionary subsets B, M, P, S, L, N, G, K, Z, T, D, A, I, U, E, O.

Y	
$B = \{balen, babi, banua\}$	$Z = \{ngan, ngatngat, ngalngal\}$
$M = \{malutu, marimla, mata\}$	$T = \{tamumu, tampaling, tau\}$
$P = \{panulu, paralaya, painaua\}$	$D = \{dalumdum, dalan, dapu\}$
$S = \{sampaga, salpantaya, salu\}$	$A = \{abe, ayup, aldo\}$
$L = \{lauen, lakuan, lala\}$	$I = \{ikat, indak, itu\}$
$N = \{nangnang, nasi, nanu\}$	$U = \{ulu, upaya, ugse\}$
$G = \{gabun, galo, gambul\}$	$E = \{ebun, eran, epika\}$
$K = \{kaue, kalma, kaladua\}$	$O = \{oneng, obra, orian\}$

**Words are written in Sulat Wawa*

This short dictionary of Kapampangan words will then be used to relate the consonant and vowel sounds to each set of words corresponding to the alphasyllabic sound.

III. Amanu Decision Logic

After defining the sets, the logic of the Amanu can be defined.

Let x: x is an alphasyllable of set F.

y: y is a dictionary subset within set Y, $y \subset Y$.

a: a is a word that exists in the domain of the dictionary sets of set Y.

Defining through Representation by Functions

The logic of the Amanu can be expressed through an injective function even when multiple outputs are possible. This is because in one input, Amanu can only output one and not multiple.

$$f: F \rightarrow Y$$

This means that the alphasyllables in *F* maps to the dictionary subsets of *Y*

$$f(x) = y, \text{where } a \in y$$

From the mappings of *F* to *Y*, this means that alphasyllable *x* corresponds to dictionary subset *y* where the generated result *a* is an element of *y.*

Defining through Representation by Quantifications

Let Q be the function *f: F* ⟶ *Y*

$$\forall a \exists x \exists y (Q(x, y) \wedge a \in y)$$

Meaning, for every *a*, there is one alphasyllable (consonant or vowel) in *F* and one dictionary subset in *Y* such that *x* maps to *y* and *a* is an element of *y*.

Logic and Sample Programming Representation

From the definitions above, the decision logic can be expressed through an implication.

Let x: x is the user input.

Let y: y is the alphasyllable from F.

Let V: x = y or "the user input is equal to the alphasyllable in F."

Let W: "The word in the dictionary subsets corresponds to the user input."

$$V \rightarrow W$$

I	**W**	***I → W***
T	*T*	*T*
T	*F*	*F*
F	*T*	*T*
F	*F*	*T*

Analyzing the truth table for the implication, several conclusions can be reached.

Implication Truth Values	**Conclusion**
$T \rightarrow T = T$	Correct, since inputting a syllable equal to one of the alphasyllables in set F implies generating one of the words in the dictionary subset corresponding to the alphasyllable. ex.) The user wants to generate a word with input "ba." $f(ba) = B$ $B = \{balen, babi, banua\}$ $f(ba) = balen$
$T \rightarrow F = F$	This is an example of a logical error. If the input is equal to an alphasyllable yet no words were generated corresponding to the input, then the program of the language model has an error. ex.) The user wants to generate a word with input "na." $f(na) = K, na \neq ka\ yet\ na \rightarrow K$ $K = \{kaue, kalma, kaladua\}$ $f(na) = kaladua$
$F \rightarrow T = T$	In this result, it is an example of input validation. It can refer to the same input, but with different cases, that is when the input is lowercase or uppercase. ex.) The user wants to generate a word with input "BA." $f(BA) = B, BA \neq ba\ yet\ BA \rightarrow B$ $B = \{balen, babi, banua\}$ $f(BA) = balen$
$F \rightarrow F = T$	Correct, since the wrong input leading to the wrong output shows that the decision logic is still correct. ex.) The user wants to generate a word with input "ba," but inputted the wrong syllable, "pa." $f(pa) = P$ $P = \{panulu, paralaya, painaua\}$ $f(pa) = paralaya$

Assuming $V \rightarrow W = T$, a Java pseudocode can be made to represent the conditional statements wherein the exclusive or ($\oplus$) symbol is used for the output since the generative model must not output two words at the same time as probabilities cannot be equal.

If input.equalsIgnoreCase("ba") → $B = \{balen \oplus babi \oplus banua\}$

If input.equalsIgnoreCase("ma") → $M = \{malutu \oplus marimla \oplus mata\}$

If input.equalsIgnoreCase("pa") → $P = \{panulu \oplus paralaya \oplus painaua\}$

If input.equalsIgnoreCase("sa") → $S = \{sampaga \oplus salpantaya \oplus salu\}$

If input.equalsIgnoreCase("la") → $L = \{lauen \oplus lakuan \oplus lala\}$

If input.equalsIgnoreCase("na") → $N = \{nangnang \oplus nasi \oplus nanu\}$

If input.equalsIgnoreCase("ga") → $G = \{gabun \oplus galo \oplus gambul\}$

If input.equalsIgnoreCase("ka") → $K = \{kaue \oplus kalma \oplus kaladua\}$

If input.equalsIgnoreCase("nga") → $Z = \{ngan \oplus ngatngat \oplus ngalngal\}$

If input.equalsIgnoreCase("ta") → $T = \{tamumu \oplus tampaling \oplus tau\}$

If input.equalsIgnoreCase("da") → $D = \{dalumdum \oplus dalan \oplus dapu\}$

If input.equalsIgnoreCase("a") → $A = \{abe \oplus ayup \oplus aldo\}$

If input.equalsIgnoreCase("i") → $I = \{ikat \oplus indak \oplus itu\}$

If input.equalsIgnoreCase("u") → $U = \{ulu \oplus upaya \oplus ugse\}$

If input.equalsIgnoreCase("e") → $E = \{ebun \oplus eran \oplus epika\}$

If input.equalsIgnoreCase("o") → $O = \{oneng \oplus obra \oplus orian\}$

IV. Representing Input and Output Through Relations

Through relations, the conditional statement and pseudocode can be represented through a set *R* containing subsets in the cartesian product of the input set *F* and a dictionary subset from *Y* where the relation's ordered pair is defined as *(input, output) = (x, y),* where *x* is the input and *y* is the output. Thus, the relation can be defined as such.

Let R_n: R_n is the set containing the relations.

Q: Q is a dictionary subset such that $Q \in Y$

$$R_n \subseteq F \times Q$$

Thus, the relations are shown as $R_n = \{(x \in F, y \in Q), (x \in F, y \in Q), (x \in F, y \in Q)\}$.

$R_1 = \{(ba, balen), (ba, babi), (ba, banua)\}$

$R_2 = \{(ma, malutu), (ma, marimla), (ma, mata)\}$

$R_3 = \{(pa, panulu), (pa, paralaya), (pa, painawa)\}$

$R_4 = \{(sa, sampaga), (sa, salpantaya), (sa, salu)\}$

$R_5 = \{(la, lauen), (la, lakuan), (la, lala)\}$

$R_6 = \{(na, nangnang), (na, nasi), (na, nanu)\}$

$R_7 = \{(ga, gabun), (ga, galo), (ga, gambul)\}$

$R_8 = \{(ka, kaue), (ka, kalma), (ka, kaladua)\}$

$R_9 = \{(nga, ngan), (nga, ngatngat), (nga, ngalngal)\}$

$R_{10} = \{(ta, tamumu), (ta, tampaling), (ta, tau)\}$

$R_{11} = \{(da, dalumdum), (da, dalan), (da, dapu)\}$

$R_{12} = \{(a, abe), (a, ayup), (a, aldo)\}$

$R_{13} = \{(i, ikat), (i, indak), (i, itu)\}$

$R_{14} = \{(u, ulu), (u, upaya), (u, ugse)\}$

$R_{15} = \{(e, ebun), (e, eran), (e, epika)\}$

$R_{16} = \{(o, oneng), (o, obra), (o, orian)\}$

Moreover, these relations are antisymmetric as there is no reverse ordered pair – meaning that the ordered pair *(input syllable, output word)* has no corresponding *(input word, output syllable)* pair as Amanu only focuses on predicting a word based on a syllable, not predicting a syllable based on the word. In proving this, an antisymmetric relation is defined such that for a pair $(a, b) \in F \times Q$: $\forall a \forall b(((a, b) \in R \wedge (b, a) \in R) \rightarrow b = a)$ Meaning that when a pair (a,b) and its reverse (b, a) is an element of the defined relation, then a and b are equal.

Take $R_{11} = \{(da, dalumdum), (da, dalan), (da, dapu)\}$, every ordered pair can be plugged into the antisymmetric definition of the relation to prove that every relation defined as $R_n = \{(x \in F, y \in Q), (x \in F, y \in Q), (x \in F, y \in Q)\}$ is antisymmetric.

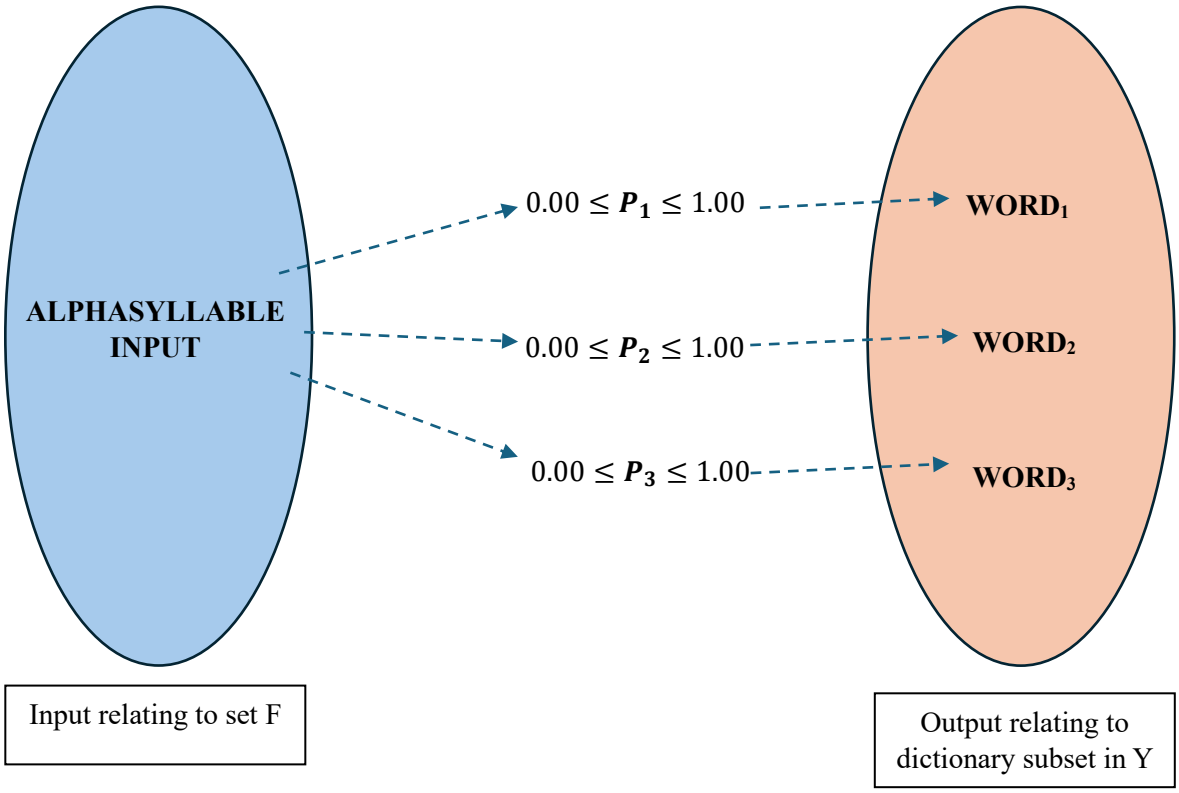
	Ordered Pair (a, b)	**Reverse Pair (b, a)**	**Equality b = a**
	$(da, dalumdum)$	$(dalumdum, da)$	$dalumdum = da$
Test	$(da, dalumdum) \in R?$	$(dalumdum, da) \in R?$	$dalumdum = da?$
Result	T	F	F
Truth Value	$(T \wedge F \rightarrow F) = \boldsymbol{T}$		
	Ordered Pair	**Reverse Pair**	**Equality**
	$(da, dalan)$	$(dalan, da)$	$dalan = da$
Test	$(da, dalan) \in R?$	$(dalan, da) \in R?$	$dalan = da?$
Result	T	F	F
Truth Value	$(T \wedge F \rightarrow F) = \boldsymbol{T}$		
	Ordered Pair	**Reverse Pair**	**Equality**
	$(da, dapu)$	$(dapu, da)$	$dapu = da$
Test	$(da, dapu) \in R?$	$(dapu, da) \in R?$	$dapu = da?$
Result	T	F	F
Truth Value	$(T \wedge F \rightarrow F) = \boldsymbol{T}$		

With this, Amanu's logic only extends in a way that for every relation, it will only accept a syllable input rather than a word input as its logic imposes a strict rule of "predicting a word based on the syllable." Furthermore, the relations also preserve the language model's function as a generative AI, relying on probabilities for output – thus needing multiple output options.

V. Output Generation with Probabilities and Matrices

Through the generation of an output through an input as represented in the previous relations, the output can be generated using probabilities. Mapping x as input to y as output, there is a probability of less than or equal to 100% for each output in the relation set, where the sum of all the probabilities is 100%. To visualize, it can be represented graphically:

Let P: P is the probability of generating the word output less than or equal to 100%.



where $P_1 + P_2 + P_3 = 1.00$

Subsequently, the language model outputs the word that has the highest probability wherein $P_{output} > P_n \geq P_n$ such that Amanu can only output one word.

To find this highest probability to output the word, Amanu can use basic matrix multiplication where it will be based on two matrices (an input matrix and a weight matrix) that are multiplied together to get a resulting matrix containing the numerical scores of each candidate word in the dictionary subset. These scores represent how probable and how strong each word is predicted as the next output.

To convert the scores and get the probability values, the scores are summed up, and each score is divided by the total. This is a process of normalization that produces a probability value of between 0 and 1 for each possible word output. The word with the highest percentage will then be selected and output as the result of the input.

Additionally, it is important to note that the normalization process in real-life, larger AI language models use a function called a *softmax* function, converting raw output scores from a neural network to a series of probabilities between 0 and 1 in a probability distribution.

However, in this case study, the aforementioned process of using matrix multiplication will be used to simulate this normalization process and get the highest probability to output a word.

Matrix Representation and Formulas

Let I: I is the input matrix.

O: O is the weight matrix.

S: S is the score matrix.

w: w is the weight of the alphasyllable input and candidate word.

s: s is the score of each word after the matrix multiplication.

For the alphasyllable input, it can be represented by a 1×2 matrix as a vector where the row is the alphasyllable's numerical meaning or weight.

$$I = [w_{i.1} \quad w_{i.2}]$$

For the matrix for the output containing candidate words, it can be represented by a 2×3 matrix where each column containing two weights represents a candidate output word.

$$O^T = \begin{bmatrix} w_{o.1.1} & w_{o.2.1} & w_{o.3.1} \\ w_{o.1.2} & w_{o.2.2} & w_{o.3.2} \end{bmatrix}$$

$$where$$

$$O^T = \begin{bmatrix} w_{o.1.1} & w_{o.2.1} & w_{o.3.1} \\ w_{o.1.2} & w_{o.2.2} & w_{o.3.2} \end{bmatrix}$$

$$word_1 \quad word_2 \quad word_3$$

The resulting matrix containing the scores of each candidate word in relation to the input matrix will then be represented by a 1×3 matrix.

$$S = [s_1 \quad s_2 \quad s_3]$$

Each score can be calculated as:

$$s_1 = w_{i.1}w_{o.1.1} + w_{i.2}w_{o.1.2}$$

$$s_2 = w_{i.1}w_{o.2.1} + w_{i.2}w_{o.2.2}$$

$$s_3 = w_{i.1}w_{o.3.1} + w_{i.2}w_{o.3.2}$$

For the normalization process, it can be represented as a formula where one score from the score matrix is divided by the sum of all scores in the score matrix. This will result in a value of between 0 and 1, giving the probability of the candidate word to be outputted.

$$P_n = \frac{s_n}{s_1 + s_2 + s_3}$$

AI Learning and Weight Re-Calculation

The AI language model's weight matrix, at first, is a collection of random values that become learned parameters to create a weight matrix when the AI "trains" itself to learn and output the desired values wherein these weight matrices are not arbitrary.

These weights in the weight matrix are first initialized into small random values within a specified range. Then, throughout training, these values are continuously adjusted whenever an error occurs to reduce the chance for errors and approach the correct output. Furthermore, each weight continuously becomes less random during training as the AI starts to learn patterns, comparing previous iterations to its new outputs until the correct and desired answer is obtained.

Basically, it could be broken down into four steps:

1. The weight matrix starts as a random collection of numbers within a certain range.
2. The AI makes an error in its output.
3. The AI adjusts the values of the numbers in the matrix.
4. If the output is correct, the AI adjusts the matrix to make the output more probable to be chosen as the output; if not, the AI adjusts the values again.

In visualizing these steps, it starts with a random set of weights in the weight matrix often within a small range to keep the values stable.

Let weight matrix O has numerical values in range:

$$-1.00 \leq w \leq 1.00$$

The weight matrix can start off as:

$$O = \begin{bmatrix} -0.15 & 0.12 & 0.45 \\ 0.04 & 0.50 & -0.67 \end{bmatrix}$$

Through gradient descent, the language model continuously iterates its parameters, the word weights in this case, to minimize the error. It does this through a series of steps:

1. Make a prediction for an output.
2. Compare the calculated prediction with the correct output.
3. Calculate the degree of error between the prediction and correct output.
4. Re-adjust the weights to minimize the error.

Doing so, the final weight matrix can look like this:

$$O^T = \begin{bmatrix} w_{o.1.1} & w_{o.2.1} & w_{o.3.1} \\ w_{o.1.2} & w_{o.2.2} & w_{o.3.2} \end{bmatrix}$$

$$\downarrow$$

$$O^T = \begin{bmatrix} 0.50 & 0.45 & 0.29 \\ 0.36 & 0.72 & 0.88 \end{bmatrix}$$

These makes the language model predict words better as the error becomes minimized.

Calculating the Probabilities and Determining Output

In obtaining this less error prone weight matrix, the input matrix and the weight matrix can now be multiplied to get the score matrix.

If the user inputs “sa”, then the possible outputs become the dictionary subset *S* from set Y. These words then become the possible outputs for the language model. With this, Amanu collates the weights of the input as well as the outputs.

Let $Score = sa \times S^T$.

$$sa = [0.67 \quad 0.21]$$

$$S = \{sampaga, salpantaya, salu\}$$

$$S^T = \begin{bmatrix} 0.50 & 0.45 & 0.29 \\ 0.36 & 0.72 & 0.88 \end{bmatrix}$$

After representing the input and candidate words with numerical weights, the matrices can then be multiplied to get the dot products to get the scores in the score matrix.

$$Score = [0.67 \quad 0.21] \times \begin{bmatrix} 0.50 & 0.45 & 0.29 \\ 0.36 & 0.72 & 0.88 \end{bmatrix}$$

$$s_1 = (0.67 \times 0.50) + (0.21 \times 0.36)$$

$$s_2 = (0.67 \times 0.45) + (0.21 \times 0.72)$$

$$s_3 = (0.67 \times 0.29) + (0.21 \times 0.88)$$

It results in the following score matrix.

$$Score = [0.41 \quad 0.45 \quad 0.38]$$

Now, the scores are then normalized to get the probability. In real large language models, however, it is important to note that normalization is often done with a softmax function. In the current context, Amanu will only utilize the following processes.

Formula: $P = \frac{s_n}{s_1 + s_2 + s_3}$

$$P_1 = \frac{0.41}{0.41 + 0.45 + 0.38} = 0.33$$

$$P_2 = \frac{0.45}{0.41 + 0.45 + 0.38} = 0.36$$

$$P_3 = \frac{0.38}{0.41 + 0.45 + 0.38} = 0.31$$

Result

Code	**Word**	**Probability**
P_1	*sampaga*	*0.33*
P_2	*salpantaya*	*0.36*
P_3	*salu*	*0.31*

Since $P_2 > P_1 > P_3$, Amanu will output salpantaya since it has a bigger probability (0.36) than sampaga (0.33) and salu (0.31). Through this process, it shows that matrices can be used effectively to predict an output based on a user input.

VI. Possible Updates and Set Operations

Amanu may implement some additional features in its system. These features may include word categorization and filtering that may assist the basic functionality of the language model. With the word categorization update, Amanu may be trained to sort words that are different types of nouns such as names, things, animals, places, ideas, or events based on its database, as well as specific user-defined rules. For example, take dictionary subset *D* and sets *I* for ideas.

$$D = \{dalumdum, dalan, dapu\}$$

$$I = \{dalumdum, painaua, kalma\}$$

Intersecting the sets will validate which words are included in the category. It can also define rules on the categorization, where in this case, Amanu should only output a word that is an idea and starts with the input "da."

$$D \cap I = \{dalumdum\}$$

In another example, take set *Y* and the rules indicate only words that are in the vowel output sets ($A \cup I \cup U \cup E \cup O$) and are animals are printed as output. Let *Z* also be the set containing a set of possible animals in the language model's database.

$$Z = \{dapu, ayup, asan, itu, pusa, damulag\}$$

$$(A \cup I \cup U \cup E \cup O) \cap \mathrm{Z} = \{ayup, itu\}$$

This feature could upgrade the AI to filter the output based on specific user preferences.

VII. Reflection and Integration: Kapampangan Innovation with AI

Throughout this case study, it showcases how the fundamental principles of generative AI and language models can be explained using the basic concepts of discrete mathematics. By this principle, the paper reinforces understanding of basic computing logic and artificial intelligence decision making processes as the case study provides an opportunity to explore discrete mathematical concepts beyond the classroom and integrate them into real-life applications.

The case study also presents a possible innovation in connection with Kapampangan culture amidst technological developments. As the Kapampangan language is starting to become more obsolete as generations come, AI integration in Kapampangan language learning provides a hopeful future where many can have access in learning the language. With this, the case study emphasizes that building artificial intelligence programs that empower Kapampangan culture, such as Amanu, preserves culture, innovates technology, and empowers Kapampangan talent and potential for computing.

Furthermore, Amanu not only shows that the course is essential in computing programs, but it also empowers students to enhance their critical, logical, and analytical thinking – especially in the computing field and in an age where almost anything can be done by AI. Thus, building understanding and comprehension regarding AI systems is essential, enabling students to be critical yet constructive in the innovation age of artificial intelligence – to see it as potential tech development, not just a “cheating tool.”

With this, the course leaves a mark on personal learning. From the topic on logic that reinforced logical thought and decision making, the sets that fostered structured and organized learning, to the relations that showcased how objects interact, and the matrices that revealed abstract thinking and AI learning, the course highlights how discrete mathematics is not purely memorization and process-based, but also an active mental and cognitive exercise that trains the mind to think creatively and logically even when faced with unfamiliar circumstances.

Thus, through this study and Amanu, artificial intelligence and Kapampangan as a language may be unfamiliar together, yet through discrete mathematics, these ultimately may come into unexpected innovation. A technological innovation, filled with culture that is empowered in the artificial intelligence age.

References

Babcock, J., & Bali, R. (2025). *Generative AI with Python and PyTorch* (2nd ed). Packt Publishing.

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic model. *Journal of Machine Learning Research 3,* 1137-1155. https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf?utm_source=chatgpt.com

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://aikosh.indiaai.gov.in/static/Deep+Learning+Ian+Goodfellow.pdf

Vaswani, A., Shazeer, N., Parmar, N., Uszkoriet, K., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). *Attention is all you need.* https://doi.org/10.48550/arXiv.1706.03762